Intro to R



Robert Gentleman **Ross Ihaka**

Creators of R

- Just about everything named in R is an object
- An object is a **container that**
    - knows its **class (label for what's inside).**
    - has contents (e.g., Actual numbers).

Intro to R: Examples of objects

- **dataset,** which you use for analysis (various classes)
- **functions,** which perform analysis (function class)
- **results,** which come out of analysis (various classes)

In effect, you always get a new dataset filled with results when you analyze data.

Classes of data values inside data objects

- **Numeric:** Continuous/discrete numeric variables
- **Factor:** Categorical variables
- **Logical:** TRUE/FALSE binary variables
- **Character:** Text

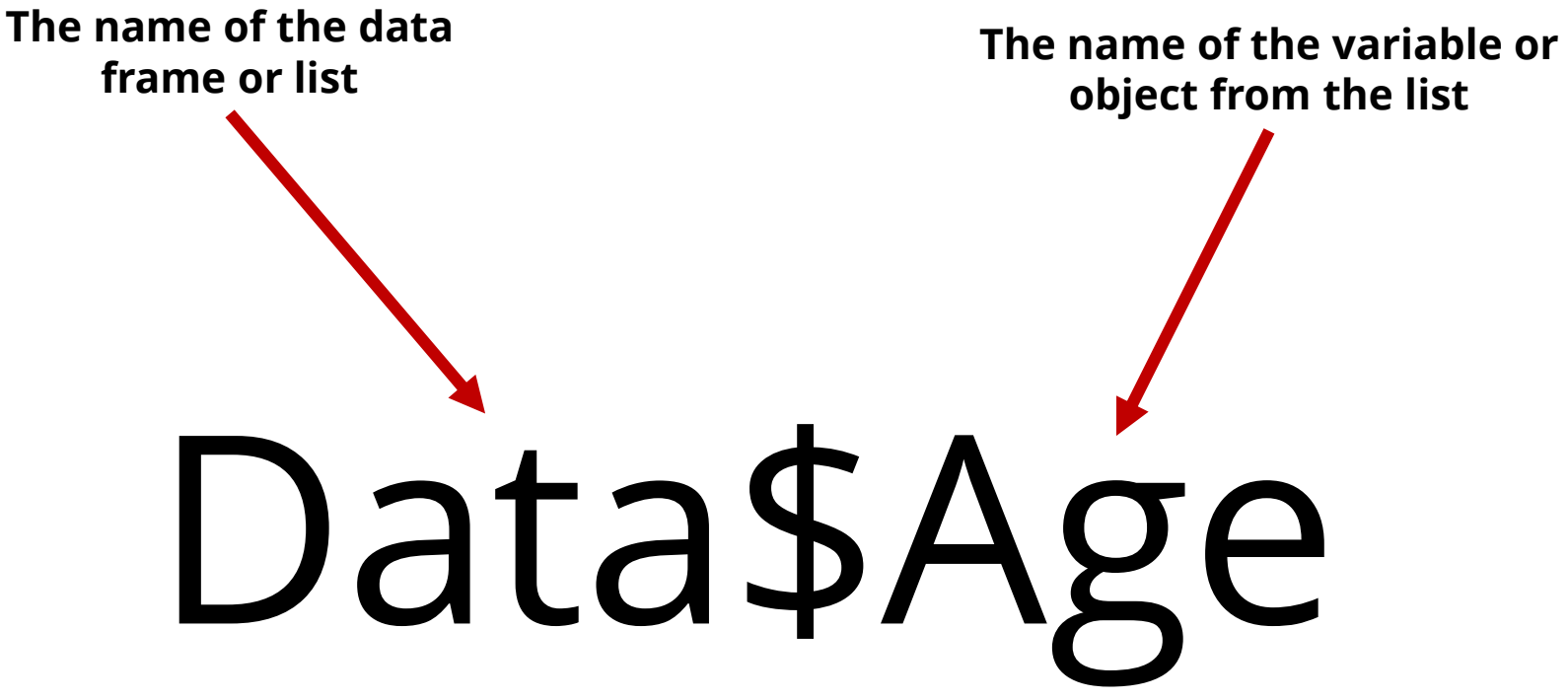
Intro to R: Class

- An object's class tells R how the object should be handled.
- For example, summarizing data should work differently for numbers and categories!

Intro to R: Types of objects/data in R

- **Vector,**
- **Matrix:** is a set of vectors with the same length and same type,
- **Data frame:** is a set of vectors with the same length but can be in different types,
- **List:** a set of objects with any type and any length.

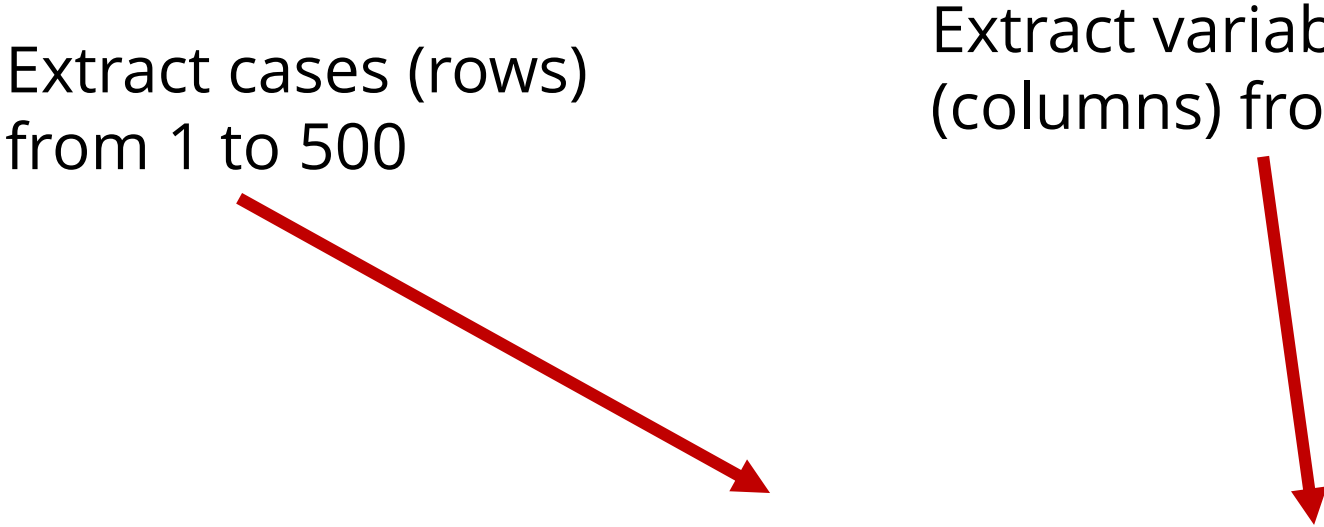
Intro to R: Access vectors and objects from the list



Intro to R: Accessing subset of the data frame

Extract cases (rows) from 1 to 500

Extract variables (columns) from 2 to 5



```
Data[1:500, 2:5]
```

Definition

Descriptive statistics is the discipline of quantitatively describing the main features of a collection of data.

Descriptive statistics is typically distinguished from inferential statistics. With descriptive statistics you are simply describing what is or what the data shows. With inferential statistics, you are trying to reach conclusions that extend beyond the immediate data alone.

#load the built in dataset mtcars
data(mtcars)

```
#Shows the structure of the object (data frame in this case)
        str(mtcars)
# shows the dimension of the object (number of cases (rows) and  number of variables
(columns))
        dim(mtcars)
# Shows the length of the object
        length(mtcars)
# Shows the number of rows and columnss
        nrow(mtcars)
        ncol(mtcars)
#Shows the firs n cases
        head(mtcars, n=1)
#shows the last n cases
        tail (mtcars, n=5)
```

Intro to R

```
#Chose the first 3 variables of the data frame and includes it in a new dataframe
        New<-mtcars[,1:3]
#Creates new dataframe with 1-20 rows and 1-3 variables of the original data frame
        New<-mtcars[1:20,1:3]
#Creates new dataframe with variables mpg, hp, wt
        New<-mtcars[,c("mpg","hp", "wt")]
#Shows the n-th row
        mtcars[10]
#Shows the number corresponding to n-th (10) row and m-th column (1)
        mtcars[10,1]
#Finds the minimum of the vector
        min(mtcars$mpg)
#Or
# Finds the number of the row associated with the minimum number of the variable, then we look at that number
        which.min(mtcars$mpg)
        mtcars$mpg[[15]]
```

Intro to R

```
#read the csv file into R
        census_csv<-read.csv("census R.csv")
#We have nominal and ordinal variables here, so we will need to set factor levels for them

#Nominal variable
        census_csv$sex<-factor(census_csv$sex, levels=c(1,2), labels=c("Male",
"Female"))
# Ordinal variable
        census_csv$happy<-ordered(census_csv$happy, levels=c(1,2,3),
labels=c("Very Happy", "Pretty Happy", "Not too Happy"), exclude=c(8,9))

#handling missing values - Age variable has recoded missing values as 98,99
We need to set them as missing values in R

        census_csv$age[census_csv$age==99]<-NA
        census_csv$age[census_csv$age %in% c(98,99)] <- NA

#Lets look at the variable
        str(census_csv$age)
```

Min and Max

Sample maximum and sample minimum, also called the largest observation, and smallest observation, are the values of the greatest and least elements of a sample.

Data: 1 3 5 2 9
Min: 1
Max: 9

R example

```
> x<-c(155, 160, 171, 182, 162, 153, 190, 167, 168, 165, 191)
min(x)
[1] 155
```

Mean

The sample mean is the arithmetic average of the data.
It can be calculated, by summing all of the data values and dividing the sum by the total sample size.

Data: 1 3 5 2 9
Mean: (1+3+5+2+9)/5=20/5=4

Mathematically: for a variable X, mean is often denoted as x and calculated as:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \ldots + x_n}{n}$$

R example

In R to calculate the of sequence mean used “mean” command:

```
x <- c(1,3,5,2,9)
> mean(x)
> [1] 4
```

If there are missing values:

```
x <- c(1,3,5,2,9,NA,7,10)
> mean(x)
[1] NA
> mean(x, na.rm=T)
[1] 5.285714
```

Median

Sometimes it is of interest to sort values of a variable in ascending or descending order.

Median is the middle point of ordered data – either the middle observation (if the number of observations is odd) or the average of the two middle observations (if the number of observations is even).

Example:
Body heights of 11 individuals (in cm):

155, 160, 171, 182, 162, 153, 190, 167, 168, 165, 191. Ordered data:
median: 167

R example: Median

In R to calculate the of sequence median used “median” command:

```
> x<-c(155, 160, 171, 182, 162, 153, 190, 167, 168, 165, 191)
> median(x)
[1] 167

> x<-c(155, 160, 171, 182, 162, 153, 190, 167, 168, 165, 191, 175)
> # added 1 observation
> median(x)
[1] 167.5
```

Range

The range is simply the highest value minus the lowest value.

Data: 1 1 5 2 9 8 7
Range: 9-1 = 8

R example

- x<-c(155, 155, 155, 182, 162, 153, 190, 167, 168, 165, 191)

range(x)

```
range(x)[2]-range(x)[1]
```

Standard Deviation

Standard deviation (SD, s) is a quantity that reflects the variability of the sample. One could interpret SD as the approximate mean distance from the mean.
The Standard Deviation shows the relation that set of scores has to the mean of the sample. More precisely, SD is defined as the square root of the variance. (sum of squared differences from the mean divided by sample size minus 1 (the latter called as the sample variance).

$$s = \sqrt{s^2} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

R example

Similarly to the mean, standard deviation is sensitive to the extremes in the data.

```
> x<-c(1,4,5,7,8,11)
> mean(x)
[1] 6
> median(x)
[1] 6
> var(x)   # variance
[1] 12
> sd(x)  # standard deviation
[1] 3.464102
> x<-c(1,4,5,7,8,110)  # change the last observation  from 11 to 110
> mean(x)
[1] 22.5
> median(x)
[1] 6
> var(x)
[1] 1843.5
> sd(x)
[1] 42.936
```

Intro to R

```
#Lets look at basic descriptive statistics for the variables in the dataset
        summary(census_csv)
#We can look at the descriptive statistics for single variable
        summary(census_csv$age)

#We can calculate the mean for a single variable
        mean(census_csv$age)

#The command returns NA as there are NA values in the vector
#The following commands returns statistics by removing missing values
        mean(census_csv$age, na.rm=TRUE)
        median(census_csv$age, na.rm=TRUE)
        sd(census_csv$age, na.rm=TRUE)

#Calculate average by group
        aggregate(census_csv$age, by=list(census_csv$sex), FUN="mean", na.rm=TRUE)
```

```
#Average for the groups for several variables

        aggregate(census_csv[,c("age", "sibs")], by=list(census_csv$sex),
        FUN="mean", na.rm=TRUE)

        aggregate(census_csv[,c("age", "sibs")], by=list(census_csv$sex,
        census_csv$region), FUN="mean", na.rm=TRUE)

#Group means using tapply function


        tapply(X=census_csv$age, INDEX=list(census_csv$region), FUN=mean,
        na.rm=TRUE)

        tapply(X=census_csv$age, INDEX=list(census_csv$region,   census_csv$sex),
FUN=sd, na.rm=TRUE)
```

Intro to R

```
#Creates frequency table for categorical variable
        table(census_csv$sex)
#Creates contingency table for two variables
        table(census_csv$sex, census_csv$region)
#Contingency table for 3 variables
        table(census_csv$sex, census_csv$region, census_csv$wrkstat)
#Calculates proportions of each cell in the table
        my.table<-table(census_csv$sex, census_csv$region) #creates table
object
        prop.table(my.table) # % of each cell in total
        prop.table(my.table, 1) #  % of each cell in row total
        prop.table(my.table, 2) #  % of each cell in column total
```

Intro to R: basic graphs

```
#Creates bar chart for the given categorical variable
# first we create frequency table as object
        gender.distr<-table(census_csv$sex)
#Use barplot command
        barplot(gender.distr)
#Lets add some attributes
        barplot(gender.distr, legend.text=c(“Male", “Female"))
        barplot(gender.distr, ylab="Frequency", xlab="Gender")
        barplot(gender.distr, ylab="Frequency", xlab="Gender",
        main="Gender Frequency distribution")
```

Intro to R

```
#Lest use mtcars
        str(mtcars)
#Lets create scatterplot for two variables- mpg น wt
        plot(mtcars$mpg, mtcars$wt)
#Add lables to X and Y axes
        plot(mtcars$mpg, mtcars$wt, xlab="Miles per gallon", ylab="Weight")
#Add title to graph
        plot(mtcars$mpg, mtcars$wt, xlab="Miles per gallon", ylab="Weight",
                main="Correllation between weight and miles per gallon")
#Create histogram for continuous variable
        hist(census_csv$age)
        hist(census_csv$age, main="Histogram of Age")
        hist(census_csv$age, main="Histogram of Age", xlab="Age")
#Shows probability instead of frequency
        hist(census_csv$age, main="Histogram of Age", xlab="Age", freq=FALSE)
```

Intro to R: Graphs

```
# GGplot
#install and load the package ggplot
install.packages("ggplot2")
library(ggplot2)
#str of diamonds
str(diamonds)
#Lets do a random sample from the dataset
set.seed(1982)
dsmall <- diamonds[sample(nrow(diamonds), 200), ]
str(dsmall)
#create scatterplot
        qplot(carat, price, data=dsmall)
#Can do logarithmic transformation of the variable carat
        qplot(log(carat), log(price), data=dsmall)
#Can separate diamonds by the coloring on the graph
qplot(carat, price, data=dsmall, colour=color)
#separate points using shapes
        qplot(carat, price, data=dsmall, shape=cut)
# Using geom can get almost any kind of graph
        qplot(carat, price, data=dsmall, geom="smooth")
        qplot(carat, price, data=dsmall, geom=c("point","smooth"))
        qplot(log(carat), log(price), data=dsmall, geom=c("point","smooth"))
```

Intro to R

```
#Other types of graphs using geom attribute
        qplot(color, price/carat, data=diamonds, geom="jitter")
#Adding graph elements
        qplot(color, price/carat, data=diamonds, geom="jitter", main="Price per carat for colors", ylab="Price
                per Carat")
#Creates boxplot
        qplot(color, price/carat, data=diamonds, geom="boxplot")
#Creates histogram
        qplot(price, data=diamonds, geom="histogram")
#Can control the length of the bins
        qplot(carat, data=diamonds, geom="histogram", binwidth=0.1)
#Limits the length of the x axis
        qplot(carat, data=diamonds, geom="histogram", binwidth=0.2, xlim=c(0,3))
        qplot(price, data=diamonds, geom="density")
#Creates density function
        qplot(carat,data=diamonds, geom="density")
#Density function by diamond's color
        qplot(carat,data=diamonds, geom="density", colour=color)
#Histogram by color
        qplot(carat, data=diamonds, geom="histogram", fill=color, binwidth=0.2)
#Barchart
        qplot(color, data=diamonds, geom="bar")
#faceting
        qplot(carat, data=diamonds, geom="histogram", facets=color~.)
#Make it better looking
        qplot(carat, data=diamonds, geom="histogram", facets=color~.,binwidth=0.1, xlim=c(0,2))
```